Threads:

- Java provides built-in support for multithreading.
- Multithreading is a Java feature that allows concurrent execution of two or more parts of a program for maximum utilization of CPU.
- A multi-threaded program consists of two or more parts that can run concurrently.
- Each part of the program is called a thread, and each thread defines separate path of execution.
- So, threads are light-weight processes within a process.
- Light Weight Process means is a means of achieving multitasking and shares its address space and system resources with other LWPs within the same process.
- Threads can be created by using two mechanisms :
  1. Extending the Thread class
  2. Implementing the Runnable Interface

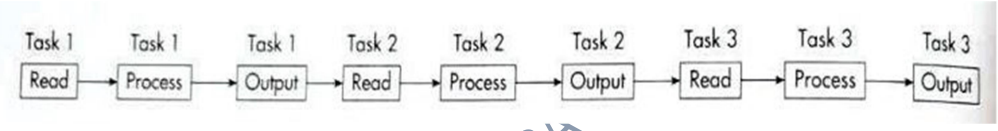
Need of Threads:

- Threads run in parallel improving the application performance.
- Each such thread has its own CPU state and stack, but they share the address space of the process and the environment.
- Threads can share common data so they do not need to use inter-process communication. Like the processes, threads also have states like ready, executing, blocked, etc.
- Priority can be assigned to the threads just like the process, and the highest priority thread is scheduled first.
- Each thread has its own Thread Control Block (TCB). Like the process, a context switch occurs for the thread, and register contents are saved in (TCB). As threads share the same address space and resources, synchronization is also required for the various activities of the thread.
- Further, increase in throughput of computer is possible only by dividing the program into segments that are data dependent and can be processed simultaneously by more than one processor; thus, it decreases the total time of computation.

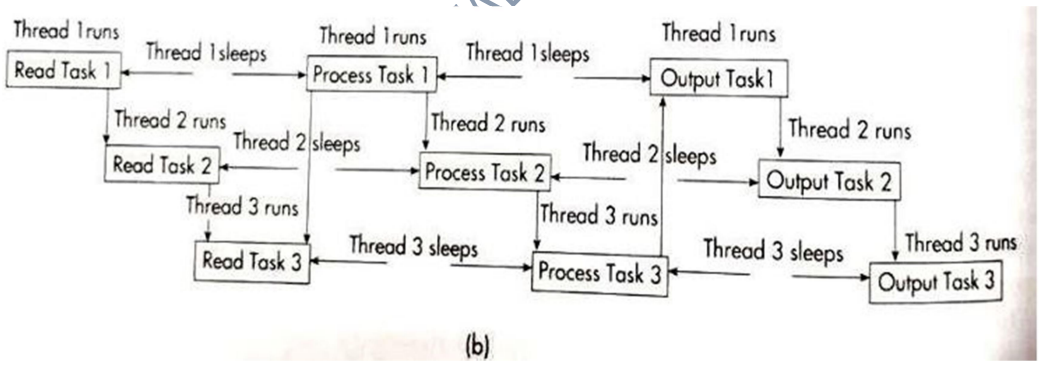
SURESH YADLAPATI

Multithreaded Programming for Multi-core Processor:

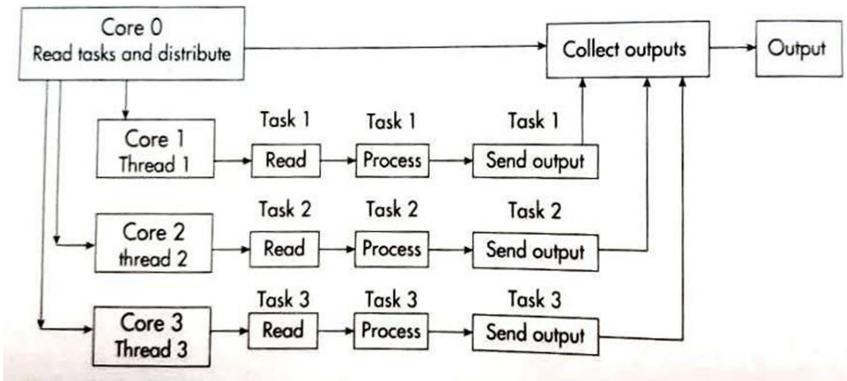
- A CPU may have two cores - dual core or four cores - quad, six cores, or more.
- CPUs having as many as 50 cores have also been developed.
- Moreover, computers with multi-core CPU are affordable and have become part of common man's desktop computer.
- Advancements in hardware are forcing the development of suitable software for optimal utilization of the processor capacity. Multithread processing is the solution.
- Multithread programming is inbuilt in Java and CPU capacity utilization may be improved by having multiple threads that concurrently execute different parts of a program.
- How Single Processor – tasks carried out in Single Sequence is illustrated in the following diagram.



- The following diagram shows the Single processor – threads share the time of processor



- The following diagram shows the Multi-core processor – threads execute concurrently



Thread Class:

- In Java, threads are based on the class Thread belonging to java.lang package, that is, java.lang.Thread.
- A thread is an object of class Thread and can invoke the instance methods defined in the class.
- Even if a thread is not explicitly defined, one thread is automatically created by the system for the execution of the **main** method. It is generally called main Thread.
- A thread does not start working on birth. It has to invoke the start() method that gives it the life, otherwise it is a lifeless thread object.
- After getting life, a thread executes a code of instructions (target) as specified by the user in the overloaded method run().

Constructors of Thread class:

- **Default Constructor:**
  - Thread t = new Thread();
    - This creates a new thread with no specific name (by default names starts from Thread-0, Thread-1 and so on) .
- **Thread with Name:**
  - Thread t = new Thread(String name);
    - This creates a new thread with a specified name
- **Thread with Runnable Target:**
  - Thread t = new Thread(Runnable target);
    - This creates a new thread that will execute the code defined in the Runnable interface's run method.
    - The name by the compiler as Thread-0, Thread-1, and so on.
- **Thread with Runnable Target and Name:**
  - Thread t = new Thread(Runnable target, String name);
    - This creates a new thread with a specified Runnable target and a name for the thread
- **Thread with Group, Runnable Target:**
  - Thread t = Thread (ThreadGroupgroup, Runnable object)
    - It allocates thread with specified group and target.
  - Thread t = Thread thread = new Thread(ThreadGroup group, Runnable target, String name);
    - This creates a new thread that belongs to a specified thread group, with a Runnable target and a name

SURESH YADLAPATI

Methods in Thread Class: The class Thread contains several methods that control the execution as well as for setting and getting attributes of threads. **All are public methods**.

Method Signature	**Description**
String getName()	Retrieves the name of running thread
final void setName(String name)	Set the name to a thread.
void run()	This method is the entry point of the thread. Execution of thread starts from this method.
void start()	This method will start a new thread of execution by calling run() method of Thread.
void sleep(long ms) void sleep(long ms, int ns)	This method suspend the thread for mentioned time duration in argument (sleep time in milli and nano seconds)
boolean isAlive()	This method will check if thread is alive or dead
Thread.State getState()	Returns the state associated with the thread
int getPriority()/ void setPriority (int value)	Obtains/sets thread priority.
final vold join() final void join(long milliseconds, int nanoseconds)	This method used to queue up a thread in execution. Once called on thread, current thread will wait till calling thread completes its execution
public static Thread currentThread()	returns the current thread instance
static void yield()	The yield() method of thread class causes the currently executing thread object to temporarily pause and allow other threads to execute.
long getID()	The getId() method is used to return the thread identifier. The thread ID is a unique positive number which was generated at the time of thread creation.
boolean isInterrupted()	Returns a boolean that indicates whether the specified thread has been interrupted.
String toString()	Returns a string representation of this thread, including the thread's name, priority, and thread group
boolean isDaemon()	is used to check if the current thread is a daemon thread.
ThreadGroup getThreadGroup()	The getThreadGroup() method in Java returns the thread group to which a thread belongs

SURESH YADLAPATI

Deprecated Methods:

The following methods are deprecated because of unsafe or are deadlock pruned.

1. stop() 2. Destroy() 3. Suspend() 4. Resume() 5. countStatckFrames()

Thread Group:

- The thread is made a member of a group at the time of its creation with constructor.
- Every thread is in fact a member of a thread group.
- If a thread's group is not specified in its constructor, as is the usual, the thread is entered into the same group as the thread that created it.
- The default thread group for a newly executing Java application is the **main** group.
- The thread groups are useful for invoking methods that apply to all the threads.
- The thread remains the member of the designated group throughout its life.

Examples of thread creation using constructors:

1. Example of Default and Parameter Constructor (thread name):

```
class Thread_Default_Parameter
{
        public static void main(String arg[]){
                Thread t1=new Thread();
                Thread t2=new Thread("PVPSIT");
                System.out.println("Current Thread:"+Thread.currentThread());
                System.out.println("Child Thread t1:"+t1);
                System.out.println("Child Thread t2:"+t2);

        }
}
```

SURESH YADLAPATI

Output:
Current Thread:Thread[#1,main,5,main]
Child Thread t1:Thread[#20,Thread-0,5,main]
Child Thread t2:Thread[#21,PVPSIT,5,main]

2. Example on Runnable

```
class MyRunnable implements Runnable {
   public void run() {
      System.out.println("Thread is running");
   }
}

public class ThreadClass_Runnable {
        public static void main(String[] args) {
           Thread t  = new Thread(new MyRunnable(), "PVPSIT");
```

Output:
PVPSIT
Thread is running

```
            t .start(); // Start the thread
            System.out.println(t .getName());
        }
}
```

3. Example program on thread groups:

```
class TG1 implements Runnable{
   public void run() {
       System.out.println(Thread.currentThread().getName());
   }
}
class TG2 implements Runnable{
   public void run() {
       System.out.println(Thread.currentThread().getName());
   }
}
public class ThreadGroupDemo{
  public static void main(String[] args) {
       ThreadGroup tg1 = new ThreadGroup("Thread Group1");
       ThreadGroup tg2 = new ThreadGroup("Thread Group2");
       System.out.println("Thread Group Name: "+tg1.getName());
                        Thread t1 = new Thread(tg1, new TG1(),"one");
       t1.start();
       Thread t2 = new Thread(tg1, new TG1(),"two");
       t2.start();

       System.out.println("Thread Group Name: "+tg2.getName());
       t3 = new Thread(tg2, new TG2(),"three");
       t3.start();
       Thread t4 = new Thread(tg2, new TG2(),"four");
       t4.start();
    }
}
```

SURESH YADLAPATI

Output:

```
Thread Group Name: Thread Group1
one
two
Thread Group Name: Thread Group2
three
four
```

Main Thread:

- When we run a program in Java, one thread is automatically created and it executes the main method. The thread is called main thread. This is the thread from which other threads are created.
- The threads created from the main thread are called child threads.
- Main thread should be the last thread to finish so that the program finishes when the main method finishes.
- The main thread can also be controlled by methods of Thread class through its reference that can be obtained by using the static method current Thread().

  Its signature is

  Thread t = Thread.currentThread();

Example:

```
class CurrentThread
{
        public static void main(String arg[]) throws Exception
        {
            Thread t=Thread.currentThread();
            System.out.println("Current Thread:"+t);
            System.out.println("Current Thread Name:"+t.getName());
            t.setName("PVPSIT");
            System.out.println("After Changing - Thread Name:"+t.getName());
            t.setPriority(2);
            System.out.println("After Changing of Priority:"+ t.getPriority());
        }
}
```

Output:

```
Current Thread:Thread[#1,main,5,main]
Current Thread Name:main
After Changing - Thread Name:PVPSIT
After Changing of Priority:2
```

SURESH YADLAPATI

Creation of New Threads:

As per Java 8, threads can be created in the following ways:

i. By extending Thread class

ii. By implementing Runnable interface: This may be carried out in the following four styles:

a. Conventional code
b. Lambda expression
c. Method reference
d. Anonymous inner class

i. Creating Thread by extending Thread class:

Procedure 1:

- Create a class that extends Thread class. (Ex. MyClass)
- In this class, overwrite(override) run() method.
  public void run() { }
  - By default this run() method is recognized and executed by the JVM
  - This is the entry point for thread
- Inside of Main method class, create an object to MyClass.
- Run the thread, for this purpose we have to use start() method.
  - MyClassobject.start();

Example:

```
/* Create a class that extends Thread class */
class MyClass extends Thread
{
        /* In this class, override run() method */
        public void run()
        {
                for(int i=0;i<200;i++)
                {
                        System.out.println(i);
                 }
        }
}
class Thread_Extends
{
        public static void main(String arg[])
        {
          /*Create an object to  MyClass */
                MyClass mc = new MyClass();
                mc.start();
        }
}
```

SURESH YADLAPATI

Procedure 2:

- Create a class that extends Thread class. (Ex. MyClass). Since this is a subclass, it inherits the methods of Thread class.
- This class calls a constructor of Thread class from its constructor.
- In this class, override run() method of Thread class.
- Inside of the MyClass constructor call start() method to start run() method.

```
/* Create a class that extends Thread class */
class MyClass extends Thread
{
        MyClass()
        {
                start();
        }
        /* Entry point for thread */
        public void run()
        {
                for(int i=0;i<200;i++)
                {
                        System.out.println(i);
                }
        }
}
class Thread_Extends
{
        public static void main(String arg[])
        {
                new MyClass();
        }
}
```

SURESH YADLAPATI

ii. Creating Thread by implementing Runnable interface (Conventional):

- Create a class that implements Runnable interface. (Ex. MyClass)
- In this class, implement run() method .

```
public void run() {
}
```

  - By default this run() method is recognized and executed by the thread.
  - This is the entry point for thread
- Inside of Main method class, create an instance of the Thread class and pass your Runnable object to its constructor as a parameter as shown below
  - Thread t=new Thread(new Myclass()); //without name
  - Thread t=new Thread(new MyClass(),"Name"); //With name

- Run the thread, for this purpose use start() method.
- t.start();

Example:

```
class MyRunnable implements Runnable {
   public void run() {
      System.out.println("Thread : "+Thread.currentThread().getName());
   }
}
public class ThreadClass_Runnable {
   public static void main(String[] args) {
      Thread t  = new Thread(new MyRunnable(), "PVPSIT");
                t .start(); // Start the thread
   }
}
```

Output:

Thread : PVPSIT

SURESH YADLAPATI

Example program on thread creation using method reference, lambda expression and anonymous inner class:

The Lambda expression and method references are simplify the code for creating the threads.

Syntaxes:

Using methods reference:

```
new Thread (Class_Name::Method_Name).start();
```

Using lambda expression:

```
new Thread( () -> Method_Name  or {code to be executed}).start();
```

Using anonymous inner class:

```
new Thread(new Runnable()
                {
                 public void run()
                 {
                        Code to be executed

                 }
                }).start();
```

Example:

```
public class Thread_OtherWays
{
        public static void main(String[] args)
        {
                //Method reference
                new Thread (Thread_OtherWays::method1).start();
                //The following line is Lambda expression
                new Thread(() -> method2()).start();
                //The anonymous inner class
                new Thread(new Runnable()
                {
                  public void run()
                  {
                        method3();
                  }
                }).start();
        }
        static void method1()
        {
          System.out.println("It method reference thread.");
        }
        static void method2()
        {
          System.out.println("It is Lambda expression method thread.");
        }
        static void method3()
        {
          System.out.println("It is anonymous inner class thread.");
        }
}
```

Output:

```
It method reference thread.
It is Lambda expression method thread.
It is anonymous inner class thread.
```

SURESH YADLAPATI

Example: Creation of Multiple threads using Thread class - One thread prints even numbers and another thread prints even number.

```
 /* Create a class that extends Thread class */
class MyClass extends Thread
{
        /* In this class, override run() method */

        public void run()
        {
                for(int i=0;i<200;i++)
                {
                        try{
                                if(i%2==0)
                                        System.out.println("Child:"+i);
                                sleep(1000);
                        }
                        catch(Exception e){}
                }
        }
}
class Thread_Extends2
{
        public static void main(String arg[])
        {
                /*Create an object to  MyClass and run method is available for execution*/
                MyClass mc = new MyClass();
                mc.start();

                for(int i=0;i<200;i++)
                {
                        try{
                                if(i%2!=0)
                                        System.out.println("Parent:"+i);
                                Thread.sleep(1000);
                        }
                        catch(Exception e){}
                }
        }
}
```

SURESH YADLAPATI

Another Example: Creation of Multiple threads using Thread class - One thread prints even numbers and another thread prints even number.

```
class MyClass1 extends Thread{
    public void run() {
        for(;;){
            int i=1;
            while(i<100){
                if(i%2==0)
                    System.out.println(Thread.currentThread().getName()+i);
                i++;
                try{
                    Thread.sleep(1000);
                }catch(Exception e){}
            }
        }
    }
}
class MyClass2 extends Thread{
    public void run() {
        for(;;){
            int i=1;
            while(i<100){
                if(i%2!=0)
                    System.out.println(Thread.currentThread().getName()+i);
                i++;
                try{
                    Thread.sleep(1000);
                }catch(Exception e){}
            }
        }
    }
}
class ThreadEx_Procedure2_Multiple{
    public static void main(String ar[]){
        MyClass1 mc1=new MyClass1();
        MyClass2 mc2=new MyClass2();
        mc1.setName("Even: ");
        mc2.setName("Odd: ");
        mc1.start();
        mc2.start();
    }
}
```

SURESH YADLAPATI

Example: Creation of Multiple threads using Thread class - One thread prints even numbers and another thread prints even number using Procedure 2 of extending thread class.

```
class MyClass1 extends Thread{
    MyClass1(){
        start();
    }
    public void run() {
        for(;;){
            int i=1;
            while(i<100){
                if(i%2==0)
                    System.out.println("Even "+i);
                i++;
                try{
                    Thread.sleep(1000);
                }catch(Exception e){}
            }
        }
    }
}
class MyClass2 extends Thread{
    MyClass2(){
        start();
    }
    public void run() {
        for(;;){
            int i=1;
            while(i<100){
                if(i%2!=0)
                    System.out.println("Odd "+i);
                i++;
                try{
                    Thread.sleep(1000);
                }catch(Exception e){}
            }
        } }

class ThreadEx_Procedure2_Multiple{
    public static void main(String ar[]){
        new MyClass1();
        new MyClass2();
    }
}
```

SURESH YADLAPATI

Example: Creation of Multiple threads using Runnable interface - One thread prints even numbers and another thread prints even number

```
class MyClass1 implements Runnable{
    public void run() {
        for(;;){
            int i=1;
            while(i<100){
                if(i%2==0)
                    System.out.println("Even "+i);
                i++;
                try{
                    Thread.sleep(1000);
                }catch(Exception e){}
            }
        }
    }
}
class MyClass2 implements Runnable{
    public void run() {
        for(;;){
            int i=1;
            while(i<100){
                if(i%2!=0)
                    System.out.println("Odd "+i);
                i++;
                try{
                    Thread.sleep(1000);
                }catch(Exception e){}
            }
        }
    }
}

class Thread_Multiple_Runnable{
    public static void main(String ar[]){
        Thread t1=new Thread(new MyClass1() );
        Thread t2=new Thread(new MyClass2());
        t1.start();
        t2.start();
    }
}
```

SURESH YADLAPATI

Example Program: Write a JAVA program that creates threads by extending Thread class. First thread display "Good Morning "every 1 sec, the second thread displays "Hello "every 2 seconds and the third display "Welcome" every 3 seconds.

```
class MyClass extends Thread{
    public void run() {
        String str=Thread.currentThread().getName();
        if(str.equals("One")){
            for(;;){
                System.out.println("Good Morning ");
                try{
                    Thread.sleep(1000);
                }catch(Exception e){}
            }
        }
        else if(str.equals("Two")){
            for(;;){
                System.out.println("Hello");
                try{
                    Thread.sleep(2000);
                }catch(Exception e){}
            }
        }
        else if(str.equals("Three")){
            for(;;){
                System.out.println("Welcome");
                try{
                    Thread.sleep(3000);
                }catch(Exception e){}
            }
        }
    }
}
class Exercise8_First{
    public static void main(String ar[]){
        MyClass t1=new MyClass();
        MyClass t2=new MyClass();
        MyClass t3=new MyClass();

        t1.setName("One");
        t2.setName("Two");
        t3.setName("Three");
        t1.start(); t2.start(); t3.start();
    }
}
```

SURESH YADLAPATI

Using Runnable Interface:

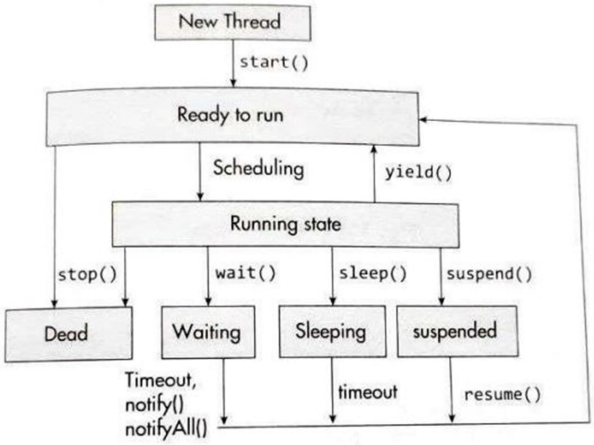
```
class MyClass implements Runnable
{
    public void run() {
        String str=Thread.currentThread().getName();
        if(str.equals("One")){
            for(;;){
                System.out.println("Good Morning ");
                try{
                    Thread.sleep(1000);
                }catch(Exception e){}
            }
        }
        else if(str.equals("Two")){
            for(;;){
                System.out.println("Hello");
                try{
                    Thread.sleep(2000);
                }catch(Exception e){}
            }
        }
        else if(str.equals("Three")){
            for(;;){
                System.out.println("Welcome");
                try{
                    Thread.sleep(3000);
                }catch(Exception e){}
            }
        }
    }
}
class Exercise8_First_Runnable{
    public static void main(String ar[]){
        Thread t1=new Thread(new MyClass(), "One");
        Thread t2=new Thread(new MyClass(), "Two");
        Thread t3=new Thread(new MyClass(), "Three");
        t1.start(); t2.start(); t3.start();
    }
}
```

SURESH YADLAPATI

Thread States:

Threads may be one of the following states:

- New thread state
- Ready-to-run state
- Running state
- Non-runnable state-waiting, sleeping, or blocked state
- Dead state



SURESHYADLAPATI

1) **New:** The thread is in new state if you create an instance of Thread class but before the invocation of start() method.

2) **Ready-to-Run (Runnable):** The thread is in runnable state after invocation of start() method, but the thread scheduler has not selected it to be the running thread.

3) Running: The thread is in running state if the thread scheduler has selected it.

4 **) Non-Runnable State:** This is the state when the thread is still alive, but is currently not eligible to run.

Sleep State: thread lies in this state until the timeout is completed or until a notification is received.

Blocked State: The thread will be in blocked state when it is trying to read input but input is not available. The thread will move from the blocked state to runnable state when it acquires input.

Wait state: The thread will be in waiting state when it calls wait() method or join() method. It will move to the runnable state when other thread will notify or that thread will be terminated.

5)Terminated : A thread terminates because of either of the following reasons: Because it exits normally.

- This happens when the code of the thread has been entirely executed by the program.
- Because there occurred some unusual erroneous event, like a segmentation fault or an unhandled exception.

Thread Priority:

Every Java thread has a priority that helps the operating system determine the order in which threads are scheduled. We can get and set the priority of a Thread. Thread class provides methods and constants for working with the priorities of a Thread.

Threads with higher priority are more important and should be allocated processor time before lower-priority threads. However, thread priorities cannot guarantee the order in which threads execute and are very much platform dependent.

The priority value is an integer ranging from 1 to 10, with three constants defined in the Thread class for setting priority levels:

Thread.MIN_PRIORITY (1) - the lowest priority
Thread.NORM_PRIORITY (5) - the default priority
Thread.MAX_PRIORITY (10) - the highest priority

Thread Priority Setter and Getter Methods

- public final void setPriority(intnewPriority) - specifies new priority to a thread
- public final int getPriority() – obtain priority value associated with the thread.

Example:

SURESH YADLAPATI

```
class Priority extends Thread {
   public Priority(String name) {
      super(name);
   }
   public void run() {
      for (int i = 1; i <= 5; i++) {
         System.out.println(Thread.currentThread().getName() + " (priority " +
                  Thread.currentThread().getPriority() + ") - Task " + i);
      }
   }
   public static void main(String[] args) {
      Priority hpt = new Priority ("High-Priority Thread");
      Priority  mpt = new Priority ("Medium-Priority Thread");
      Priority  lpt = new Priority ("Low-Priority Thread");

      hpt.setPriority(Thread.MAX_PRIORITY);    // Priority 10
      mpt.setPriority(Thread.NORM_PRIORITY); // Priority 5
      lpt.setPriority(Thread.MIN_PRIORITY);     // Priority 1
      // Start the threads
      lpt.start();
      mpt.start();
      hpt.start();
   } }
```

SURESH YADLAPATI